SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science & Engineering
ASSESSMENT TOOL 2 – TECHNICALPRESENTATION

CourseCode	**CSA0315**	**CourseTitle**	**Data Structures**
COAssessed	**CO1**	**Assessment**	**Assessment Tool 2: TECHNICAL PRESENTATION**
Weightage	**30 % of CIA**	**TotalMarks**	**100**
CO1Statement	***Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.* (BL2,BL3)**		
Student Name	**R. Kishan Raghava**	**Reg.No.**	**192525154**
Section/Batch	**Slot D / 2025**	**Date**	**06-10-2026**

Code of Conduct:

I, R Kishan Raghava(19252514), certify that this submission is my original work and that I have adheredtotheguidelinesspecifiedforthisassessment.Iunderstandthatanyviolationofacademicintegrity rules will result in disciplinary action.

Signature of the student:

R.Kishan

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts	
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established	
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure	
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence	
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand	



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Database Indexing System – Appropriate Tree Data Structure

Presented By:
R. Kishan Raghava (192525154)
CSA0315 – Data Structures

Associated professor:
Dr. Krishnaraj M



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Database Indexing System – Appropriate Tree Data Structure

Presented By:
R. Kishan Raghava (192525154)
CSA0315 – Data Structures

Presented To:
Dr. Krishnaraj M

Introduction

Database Indexing System

- Database indexing improves the speed of data retrieval.
- Large databases require efficient searching techniques.
- The indexing structure should support fast operations.
- Important operations are search, insertion, and deletion.
- Tree-based data structures are commonly used for indexing.
- B+ Tree is recommended for database indexing.

Problem and Requirements

Requirements of Database Indexing

- Fast searching of records.
- Fast insertion of new records.
- Fast deletion of existing records.
- Efficient handling of large datasets.
- Minimum disk access during operations.
- Efficient support for range queries.

Main Goal:

To select a tree structure that provides efficient database operations.

Recommended Data Structure: B+ Tree

- B+ Tree is a balanced multi-level tree.
- Each node can store multiple keys.
- Internal nodes guide the search process.
- Leaf nodes contain indexed records or record pointers.
- Leaf nodes are linked for sequential access.
- The tree remains balanced after insertion and deletion.

Structure:

Root → Internal Nodes → Leaf Nodes → Database Records

Operations and Complexity

Searching

- Start from the root.
- Compare the search key.
- Follow the appropriate child.
- Reach the required leaf node.

Insertion

- Insert the key into the appropriate leaf.
- Split the node if it becomes full.

Deletion

- Remove the key from the leaf.
- Redistribute or merge nodes when required.

Time Complexity:

- Search → **O(log n)**
- Insertion → **O(log n)**
- Deletion → **O(log n)**

Advantages

- Maintains a balanced structure.
- Provides fast search, insertion, and deletion.
- Reduces disk I/O operations.
- Handles large databases efficiently.
- Supports efficient range searching.
- Provides good scalability for database systems.

CONCLUSION

- B+ Tree is highly suitable for database indexing.
- It provides fast search, insertion, and deletion operations.
- It maintains a balanced tree structure for efficient performance.
- Linked leaf nodes make range searching faster.
- It reduces disk access and efficiently handles large databases.